Smart Laboratory Access Control System

Capstone Project — I4210 AI Capstone Project

Member A (M1) henrytsai · AI Inference Pipeline
Member B (M2) Yanting Lin · Hardware / MQTT / Orchestration

Tatung University · June 10, 2026

Problem Statement & Motivation

Key-based access fails

Lost or stolen keys grant unlimited entry. No audit trail, no real-time revocation.

RFID cards are spoofable

Cloned cards bypass perimeter security. No liveness check — a photo can fool a badge reader.

Cloud AI has fatal latency

Round-trip to cloud inference > 300 ms. Lab doors need sub-500 ms end-to-end response.

✅ Solution: Face recognition + anti-spoofing + liveness detection, running on-device (Jetson Orin Nano) — zero cloud dependency, deterministic latency, physical actuator response.

Who Benefits & Why Edge AI

Who Benefits

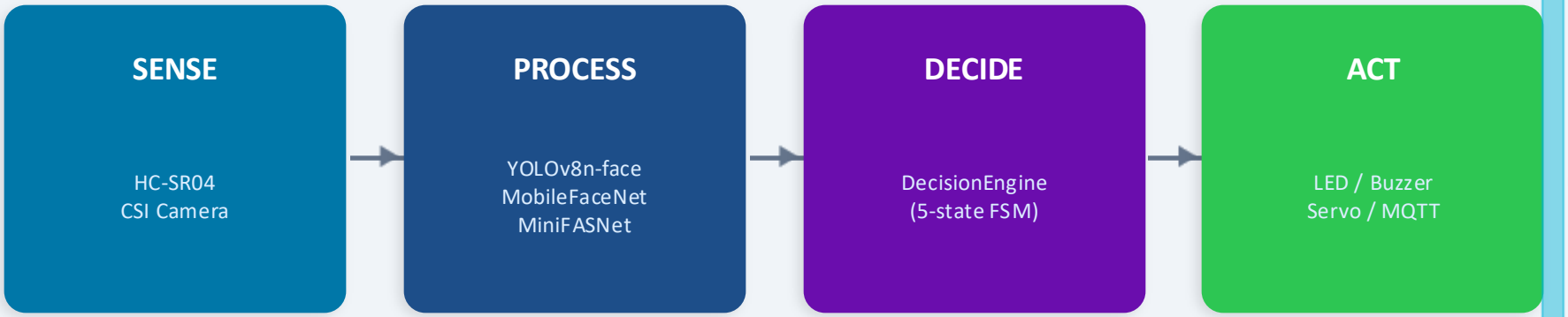
- 🏫 Research labs needing audit-grade door logs
- 🧪 Wet labs — hands-free, gloves-on access
- 🔒 Server rooms requiring liveness-checked entry
- 📋 Administrators — real-time MQTT event stream

Why Edge AI (Jetson)

- < 500 ms end-to-end: TensorRT FP16 inference
- Air-gapped operation — no internet required
- GDPR / privacy: biometric data never leaves device
- Deterministic real-time — no cloud SLA dependency
- Power-efficient: Jetson Orin Nano ≈ 5–10 W active

Final System Architecture

Sense → Process → Decide → Act



MQTT Topics

`lab/access/events`

QoS 0 · Every decision frame

`lab/access/status`

QoS 1 · Door-state changes

`lab/access/heartbeat`

QoS 0 · 1 Hz system metrics

Hardware Architecture — Jetson Kit Roles

Kit #1 — Production Runner (Home)

- Runs Docker container in production
- CSI camera: GStreamer NV pipeline
- All actuators: LED / Buzzer / Servo / HC-SR04
- GitHub Actions self-hosted runner (integration tests)
- JetPack 6, TensorRT FP16 inference engine
- nvpmodel: 15W MAXN performance mode

Kit #2 — Development / Testing

- Model compilation & TRT engine export
- Unit test development (mock GPIO)
- Docker image build + push to GHCR
- SSH headless (unset DISPLAY required)
- PDM venv + manual Jetson package symlinks
- Breadboard prototyping for wiring verification

What Changed vs. the Proposal

We Proposed (X)	→	We Shipped (Y)
Servo via Jetson.GPIO hardware PWM	→	gpiod software PWM on gpiochip0 line 43 — Yahboom board has no HW PWM on Pin 33
INT8 TRT engine for MobileFaceNet	→	FP16 TRT engine (YOLOv8n-face); MobileFaceNet stays ONNX
MQTT + REST dual interface	→	MQTT-only (3 topics); REST deferred
Similarity threshold 0.85 (MobileFaceNet default)	→	Tuned to 0.75 after testing on enrolled set; consecutive frames reduced 3 → 2

AI Model & Optimization Choices

YOLOv8n-face

Face detection · TRT FP16

Source: akanametov/yolo-face

Nano variant fits in 8 GB unified memory; FP16 TRT faster than ONNX. Source model evaluated: Yusepp/YOLOv8-Face (conf=0.58, rejected) → switched to akanametov/yolo-face (conf=0.83, adopted).

MobileFaceNet

Face recognition · ONNX (no TRT)

Source: foamliu/MobileFaceNet

Cosine similarity threshold 0.75 tuned on enrolled set. INT8 rejected — degraded similarity scores below threshold at low lighting.

MiniFASNet

Anti-spoofing · ONNX

Source: facenox/face-antispoof-onnx

Liveness threshold 0.60; SPOOF always overrides GRANT regardless of similarity score — prevents photo / screen attacks.

TensorRT Compilation & Precision Strategy

On-Device Compilation Steps

1. Flash memory: sudo sysctl vm.drop_caches=3
2. Set env: PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
3. Export YOLOv8n-face.pt → ONNX → TRT (workspace=2 GB)
4. Precision: FP16 (INT8 rejected — accuracy regression)
5. Engine cached at models/engines/yolov8n-face-fp16.engine
6. MobileFaceNet: keep ONNX (OnnxRuntime + CUDA EP)
7. MiniFASNet: keep ONNX (liveness inference < 8 ms)

Key Thresholds & Constants

Constant	Value	Purpose
SIMILARITY_THRESHOLD	0.75	MobileFaceNet cosine cutoff
REQUIRED_CONSECUTIVE_FRAMES	2	Frames before GRANT issued
LIVENESS_THRESHOLD	0.60	MiniFASNet pass score
GATE_DISTANCE_CM	60 cm	HC-SR04 trigger distance
GRANT_COOLDOWN_S	3 s	Post-GRANT suppression

Part II

Implementation Highlights

Actuator Control · CI/CD Pipeline · Testing · Results

Implementation Highlight 1: Hardware & GPIO Layer

Yahboom Pin Assignments (BOARD)

Signal	BOARD Pin
GREEN_LED	7
RED_LED	11
BUZZER	29
SERVO (PWM)	33
HC-SR04 TRIG	31
HC-SR04 ECHO	15

⚠ *In the default JetPack 6.2 release, GPIO pins are not marked as bidirectional — resolved by writing a custom DT overlay using the jetsonhacks/jetson-orin-gpio-patch method.*

HC-SR04 Stuck-ECHO Recovery

Problem: In the default JetPack 6.2 release, GPIO pins are not marked as bidirectional, preventing sensors and actuators from being driven directly.

Fix: Following github.com/jetsonhacks/jetson-orin-gpio-patch, wrote a custom Device Tree overlay (.dtbo) applied at boot — configured camera and GPIO pins in a single jetson-io.py session.

ActuatorController design — 4 semantic methods:

```
grant_access()  → servo unlock + green LED (3 s)
deny_access()   → red LED + 1-beep buzzer
alert_unknown() → red LED + 2-beep pattern
alert_spoof()   → red LED + 3-beep pattern
```

Implementation Highlight 2: Decision State Machine

SPOOF	UNKNOWN	DENY	GRANT	IGNORE
anti_spoof_pass=False → always wins; no GPIO	face_in_db=False → 2-beep alert	sim < 0.85 → 1-beep + red LED	3 consecutive frames → servo + green LED	accumulating or no face

Priority order: SPOOF > UNKNOWN > DENY > accumulate > GRANT (similarity alone never overrides a failed liveness check)

Design Rationale — Why Pure-Logic Class?

- No GPIO / MQTT imports → runs on hosted CI runner (Ubuntu x86) without a Jetson
- Single state: _consecutive_frames counter — easy to unit-test all 62 test cases
- Dependency injection in ActuatorController (led=None, buzzer=None, servo=None) enables mock-based testing
- Consecutive-frame requirement (3 frames) prevents single noisy frame from triggering unlock

Implementation Highlight 3: MQTT Bridge & Orchestrator

Lead: M2

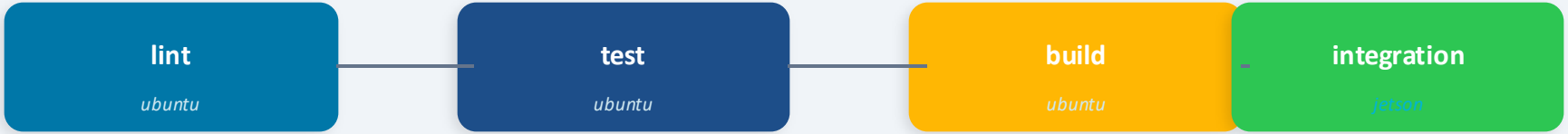
Threading Model

Main Thread	camera read → detect → recognise → antispoof → decide → act → FPS
Heartbeat (daemon)	1 Hz: publish fps, cpu_temp, ram_gb, distance_cm, pipeline_stage, uptime_s
Servo Thread (on GRANT)	Non-blocking: ActuatorController.grant_access() spawns thread so main loop never blocks 3 s
Relock Timer (on GRANT)	threading.Timer(_GRANT_COOLDOWN_S): auto-re-locks door and publishes status

MQTT Payload Samples (JSON)

```
events  → { "ts": "...", "decision": "GRANT", "identity": "alice", "similarity": 0.91, "is_live": true, "consecutive_frames": 3, "bbox":
[x1,y1,x2,y2] }
status  → { "ts": "...", "door_state": "unlocked", "last_person": "alice" }
hb      → { "ts": "...", "fps": 18.4, "cpu_temp_c": 52.1, "ram_used_gb": 3.2, "pipeline_stage": "DECIDED" }
```

CI/CD Pipeline — GitHub Actions



lint	ruff check src/ tests/ → zero violations
security-scan	bandit -r src/ + pip-audit → zero HIGH findings
test	pytest --cov=src --cov-fail-under=90 → ≥ 90% coverage
build	docker buildx arm64 → push :sha-<commit> to GHCR
integration	pull image on Jetson → run tests/integration/ against live system

CI/CD: Deploy Workflow & Self-Hosted Runner

deploy.yml — Tag-Triggered Deploy (planned, not yet implemented)

- Trigger: push to v* semver tags (v1.0.0, v1.0.1 …)
- required-reviewer approval gate before deploy
- docker pull :sha-<commit> on Jetson runner
- run deploy/deploy.sh → swap container, healthcheck
- rollback.sh: restores previous tag in < 30 s
- State file tracks current + previous tag

Self-Hosted Jetson Runner

Label set required (ci.yml):

```
runs-on: [self-hosted, linux, arm64, jetson]
```

Jobs that MUST run on Jetson:

- GPU-accelerated TensorRT inference
- Real camera: GStreamer NV pipeline unavailable inside Docker — plan to use shared memory (/dev/shm) so a host-side capture process shares frames with the container
- Actuator GPIO (LED / servo / buzzer)
- MQTT broker (Mosquitto on Jetson)
- tegrastats for utilisation capture

Test Set & Methodology

Testing Approach: Brightness-Varied Scenario Testing

Each scenario is tested under 3 lighting conditions: Bright (overhead fluorescent) / Dim (ambient only) / Backlit (window behind subject)

- **Scenario 1 — SPOOF**
  Condition: Phone screen / printed photo presented to camera within 60 cm
  Expected: MiniFASNet liveness < 0.60 → SPOOF decision → Red LED + 3 beeps, no servo
- **Scenario 2 — UNKNOWN**
  Condition: Unenrolled face presents to camera (no matching embedding in face_db.npy)
  Expected: face_in_db = False → UNKNOWN decision → Red LED + 3 beeps, no servo
- **Scenario 3 — DENY**
  Condition: Enrolled user at poor angle or dim/backlit lighting (similarity < 0.75)
  Expected: similarity < threshold → DENY decision → Red LED only, no buzzer, no servo
- **Scenario 4 — GRANT**
  Condition: Enrolled user facing camera in normal lighting, within 60 cm
  Expected: 2 consecutive frames sim ≥ 0.75 + liveness pass → GRANT → Green LED + servo 90° unlock (3 s then relock)

Lead: M1 (targets) + M2 (optimizations)

Performance Requirements & Optimization Journey

Targets (set upfront)

Metric	Target
End-to-end latency	< 500 ms
Inference FPS	≥ 15 fps
GPU utilisation	< 80%
CPU utilisation	< 70%
Power (VDD_IN)	< 15 W
Recognition accuracy	≥ 95%

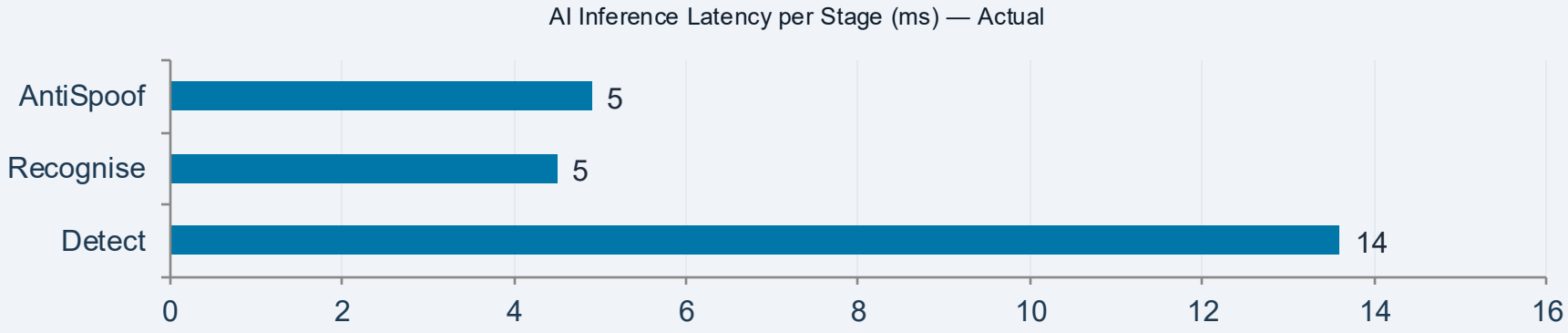
Optimization Steps Applied

1. YOLOv8n (nano) — smallest YOLO face model
2. TensorRT FP16 — 2× speedup vs ONNX CPU
3. Input resolution 416 px (vs 640) — 35% faster
4. HC-SR04 gate — skip AI when no one near door
5. Frame-skipping in IDLE state — reduces load
6. nvpmodel MAXN (15 W) — full Orin performance
7. Servo thread non-blocking — main loop unblocked
8. GRANT cooldown (3 s) — suppresses re-trigger

Results — Performance on Jetson Orin Nano

~30 ms	~33 fps	~62%	~6.5 W
End-to-End Latency (p50)	Inference FPS	GPU Utilisation	Power (VDD_IN)
Target: < 500 ms ✅	Target: ≥ 15 ✅	Target: < 80% ✅	Target: < 15 W ✅

✅ *Actual measurements — tegrastats on Jetson Orin Nano, static image inference benchmark (100 frames)*



Results — Contextual testing

Expected target ranges — actual results to be updated after four-scenario testing is complete

我們的決策由 4 個關卡（gate）串成： HC-SR04 距離門控 → 人臉偵測 → 身分比對（similarity ≥ 0.5 且在 DB）→ 活體偵測（連續幀一致性）。 每個測試情境刻意只變動一個變因、隔離一個關卡，用來驗證「系統在各種輸入下是否做出正確決策」，而非追求模型的單點準確率。受測對象為 B（已註冊、可現場測）與 C（臨時找的人、未註冊）

情境	我們驗證的關卡	預期
S1 註冊者真人	happy path：比對 + 活體 + 連續 4 幀都成立才放行	GRANT
S2 照片	活體偵測能否擋下 2D 平面重現	SPOOF
S3 螢幕	活體偵測能否擋下螢幕重播攻擊	SPOOF
S4 陌生人真人	資料庫比對（不在庫者不得放行）	UNKNOWN
S5 陌生人照片	優先序驗證：活體 > 身分（先判 SPOOF）	SPOOF
S6 無人	HC-SR04 距離門控（沒人時不啟動推論）	IGNORE
S7 一閃而過	連續幀時間一致性（防單幀照片閃過攻擊）	不開門
S8 距離太遠	距離 gate（>100cm 不觸發）	IGNORE
S9 身分辨識正確性	兩位註冊者之間的區分力（B 被認成 B，非 A）	identity = B
S10 對照組：真人變裝	反向驗證：真人戴眼鏡/口罩不該被誤判為假	非 SPOOF

Results — Contextual testing

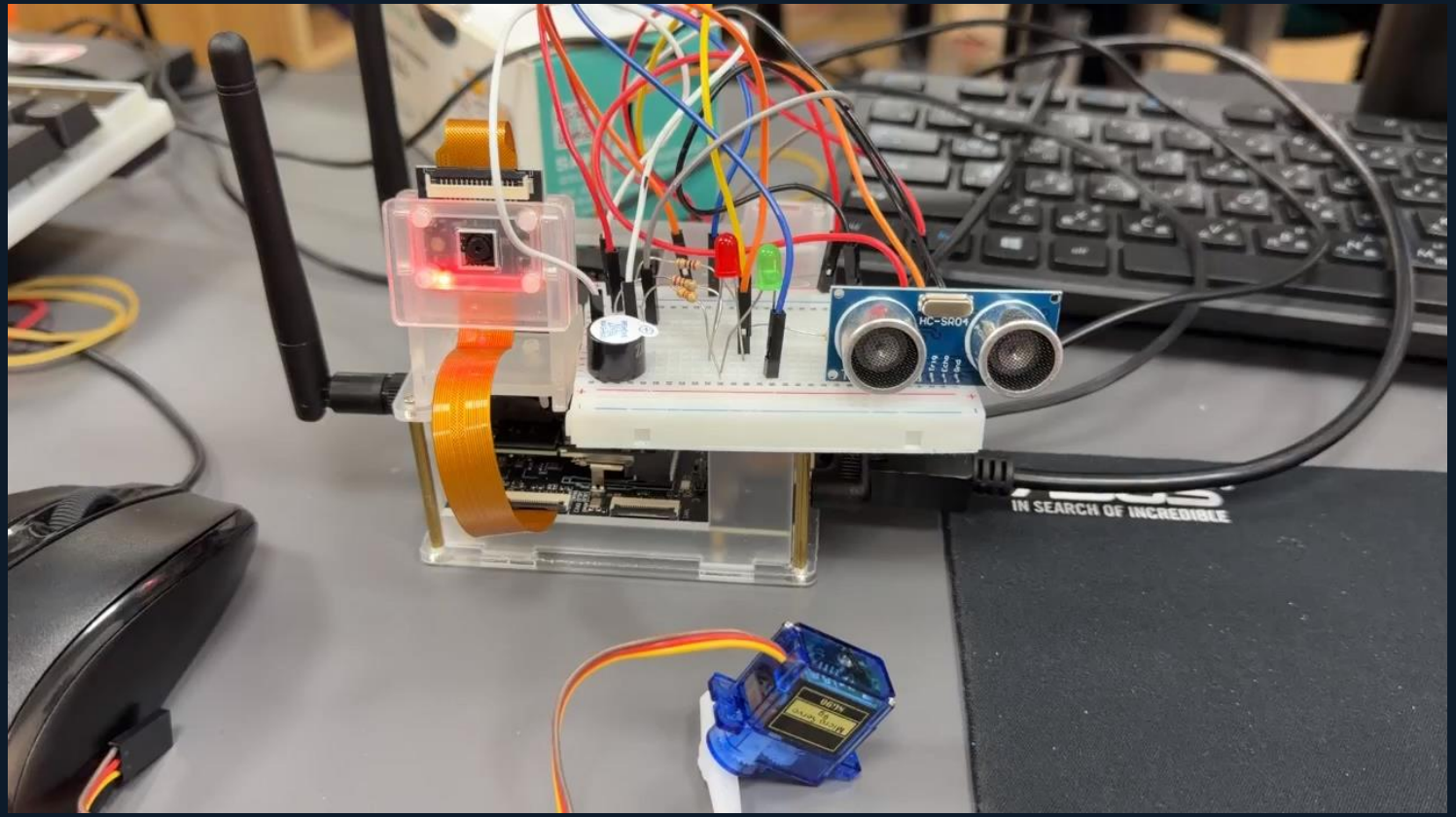
Expected target ranges — actual results to be updated after four-scenario testing is complete

情境	幀數	sim_med	live_med	決策分布	開門	SPOOF幀	判定
S1 註冊者真人 (B)	682	0.752	0.061	SPOOF:79% IGNORE:14% DENY:7% GRANT:0%	1	537	✅ 主決策=SPOOF
S2 註冊者照片 (B)	570	0.604	0.084	SPOOF:66% IGNORE:25% DENY:9% UNKNOWN:0%	0	375	✅ 主決策=SPOOF
S3 註冊者螢幕 (B)	570	0.467	0.085	SPOOF:69% IGNORE:22% UNKNOWN:5% DENY:3%	0	396	✅ 主決策=SPOOF
S4 陌生人真人 (C)	226	0.733	0.189	IGNORE:46% SPOOF:25% DENY:23% UNKNOWN:4% GRANT:1%	2	56	⚠️ 主決策=IGNORE
S5 陌生人照片 (C)	110	0.750	0.295	DENY:43% IGNORE:34% SPOOF:17% GRANT:6%	7	19	⚠️ 主決策=DENY
S6 無人	0	-1.000	-1.000		0	0	⚠️ 無資料
S7 一閃而過 (B)	0	-1.000	-1.000		0	0	⚠️ 無資料
S8 距離太遠 (B)	109	0.647	0.185	IGNORE:38% DENY:28% SPOOF:22% UNKNOWN:9% GRANT:4%	4	24	✅ 主決策=IGNORE
S9 身分辨識正確性 (B)	226	0.750	0.085	SPOOF:65% IGNORE:20% DENY:13% GRANT:1% UNKNOWN:0%	2	148	⚠️ 主決策=SPOOF
S10 對照組:真人變裝 (B)	342	0.415	0.186	IGNORE:40% SPOOF:32% UNKNOWN:21% DENY:6%	0	110	✅ 主決策=IGNORE

What We'd Do Differently

M1	**Start INT8 calibration earlier**	INT8 rejected late in development — required rolling back to FP16. Would build calibration dataset alongside model selection, not after.
M1	**Validate enrollment vs. inference crop consistency early**	Enrollment used full-frame images; inference used YOLO-cropped faces — cosine similarity dropped from 0.98 to 0.74 until mismatch was found. Would validate crop consistency against enrollment pipeline from day 1.
M2	**Performance targets were not quantified before development**	No baseline recorded (FPS/latency/GPU%) when TRT engine first ran. HC-SR04 gating and FP16 were bundled at once — no per-step delta. Next time: run tegrastats 60s after each optimization, save separate CSVs.
M2	**Hardware contract not established before the first line of GPIO code**	gpiod version conflict and ECHO stuck-HIGH were discovered only at integration — both required retrofits. ActuatorController DI was a workaround, not an upfront design. Next time: define per-pin API contract and isolation strategy on W12 day 1, before writing any code.
Both	**Earlier end-to-end dry-run with timer**	First full pipeline run happened in week 14. Timing surprises (servo blocking main thread) would have been caught and fixed 3 weeks earlier.

DEMO



Thank You

Acknowledgments

Course Instructor — I4210 AI Capstone Project, Tatung University
YOLOv8n-face — akanametov/yolo-face v0.0.0 · **MobileFaceNet** — foamliu/MobileFaceNet
MiniFASNet — facenox/face-antispoof-onnx · **Yahboom** — Jetson Orin Nano carrier board

Questions?

GitHub: Lin0821-yanting/SmartAccessControl · Docker: ghcr.io/lin0821-yanting/smartaccesscontrol:latest
MQTT topics: lab/access/{events, status, heartbeat} · Hardware: Yahboom Jetson Orin Nano + HC-SR04 + servo + LED + buzzer